

# BASIC PYTHON FOR LIFE SCIENCES

Munachiso Elekwa



# Content

## 1 Course Overview

## 2 Basic Concepts

## 3 Data Analysis with Python

# Content

## 1 Introduction

## 2 Basic Concepts

## 3 Data Analysis with Python

# Introduction

By the end of this course, you should:

- Understand basic python concepts: data types, lists, functions
- Load and explore scientific datasets
- Perform basic data cleaning
- Create simple visualizations
- Build a foundation for data science and machine learning



**WHY PYTHON?**

**OBJECTIVES**

Python is the programming language of choice in life sciences because:

- It has readable syntax and is relatively easy to learn
- It is rich in scientific and bioinformatics libraries
- It is versatile and applicable in any context: website design, data analysis, machine learning, process automation.

# Content

## 1 Course Overview

## 2 Basic Concepts

## 3 Data Analysis with Python

# Basic concepts – Statements, Comments and Strings

- A **statement** is an instruction that tells Python to perform an action.
- Statements are executed line by line from top to bottom.
- The **print()** statement is used to display output on the screen.
- **Comments** are notes written for humans and are ignored by Python during execution.
- Comments begin with the **#** symbol and are useful for explaining code.
- **Strings** are pieces of text written inside quotation marks ("" or "").
- Strings are commonly used to store names, or any text data.
- **Concatenation** means joining two or more strings together using the **+** operator.



# This is a comment

```
disease = "Malaria" #This is a string
```

```
print(disease) #Output : Malaria
```

# Concatenating two strings

```
disease = "Malaria"
```

```
print(disease + " is common in tropical regions")
```

# Output: Malaria is common in the tropical regions

# Using string methods

```
organism = "e coli"
```

```
print(organism.upper()) # displays string in uppercase
```

```
print(organism.title()) # displays string in title case
```

# Output:

E COLI

E Coli

## EXERCISE 1: Joining strings

1. Write a comment describing the code
2. Create a string variable called patient\_name
3. Create another string variable called diagnosis
4. Print both variables together in a sentence.

# Basic concepts - Variables and Data Types

**Variables** are like labeled boxes where you can store information.

- Variable names should be descriptive and meaningful (e.g., **height**, **price**,).
- Variables are assigned values using the equality =symbol.
- Variable names cannot contain spaces, use underscore instead. (eg **drug\_name** = “**Arthemeter**”)
- They make your code easier to read and organize.

Data in Python comes in different types:

- **String (str)**: A string is a sequence of characters enclosed in a single or double quotation mark
- **Integer (int)**: An Integer is a data type used to represent whole integer values
- **Float**: A float is a data type used to represent decimal values or floating point
- **Boolean (bool)**: it is used to represent the truth value of an expression. The value can only be **true** or **false**



```
# drug variables
drug_name = "Arthemeter" #This is a string
molecular_weight = 298.37 #This is a float
num_replicates = 3 #This is an integer
is_antimalarial = True #This is boolean

print(f'{drug_name} MW={molecular_weight} g/mol')
# Output: Artemeter MW=298.37 g/mol
```

# Basic concepts - Lists and list methods

A **list** in Python is an ordered collection that lets you store multiple items in a single variable.

- To create a list, use **square brackets []** and separate items with commas
- You can access items by their **index** (the position of an item in the list). Remember, Python starts counting from 0!
- Lists can be modified using several list methods:
- **.append(x)**: Adds an item to the end of the list.
- **.insert(index, x)**: Inserts an item at a specific position.
- **.remove(x)**: Removes the first occurrence of an item.



```
dose_mg = [5, 10, 20, 40, 80] #This is a list

# Indexing (zero-based)
print(dose_mg[0])    # 5 (first item)
print(dose_mg[-1])   # 80 (last item)
print(dose_mg[1:4])  # [10, 20, 40]

# Useful list operations
# Add item to the end
dose_mg.append(160)   # [5, 10, 20, 40, 80, 160]
# Add an item at specific position
dose_mg.insert(2, 160) # [5, 10, 160, 20, 40, 80]
# Return the highest number
print(max(dose_mg))   # 80
# Return the least number
print(min(dose_mg))   # 5
# Return the sum of items in a list of numbers
print(sum(dose_mg))   # 155
# Return the count of items in a list
print(len(dose_mg))   # 5
```

Output

## EXERCISE 2: The Hit List

You have IC50 values (nM) for 8 compounds: [450, 12, 3200, 87, 5, 230, 1050, 33]

1. Store these in a list called `ic50_list`.
2. Print the best (lowest) and worst (highest) IC50.
3. Sort the list and print it.
4. A new compound with IC50 = 8 nM joins the screen. Add it and re-sort.



# Basic concepts - Dictionaries

A **dictionary** in Python is a collection of **key-value pairs**. Think of it as a real dictionary where each word (key) has a definition (value).

- Use curly braces `{}` to define a dictionary. Each item is a pair: **key: value**
- To get the value for a specific key, use square brackets `[]`
- Dictionary methods include:
  - **keys()**: Returns a list of all the keys
  - **values()**: Returns a list of all the values
  - **items()**: Returns a list of all key-value pairs as tuples.
  - **del dict\_name["key"]**: Deletes the item with the specified key



```
person = { "name": "Chidi", "age": 25, "city": "Abuja" }  
#This is a dictionary
```

```
# Accessing values
```

```
print(person["name"]) # Output: Chidi
```

```
#Add new pair
```

```
person["email"] = "chidi.example.com" # { "name":  
"Chidi", "age": 25, "city": "Abuja", "email":  
"chidi.example.com" }
```

```
#Update existing value
```

```
person["age"] = 26 # { "name": "Chidi", "age": 26,  
"city": "Abuja" }
```

```
#delete a pair
```

```
del person["city"]  
print(person) # { "name": "Chidi", "age": 25 }
```

## EXERCISE 3: Patient record

Create a dictionary named `patient_record` with this data: Name: Ada, Age:20, Test: Malaria, Result: Positive

1. Print the patient's name and test result.
2. Add the hospital name
3. Update the test result to 'negative'
4. Print the statement: 'Patient [name] is [age] years old'

(This statement should be dynamic ie it should change if you change the values)

# Basic concepts – Loops and Conditionals

## If/else (Conditional)

- An **if-else** statement is used to make decisions in Python.
- It allows the program to perform different actions depending on whether a condition is true or false.

## For Loop

- A **for loop** is used to repeat an action for every item in a list or collection.
- It helps automate repetitive tasks such as analysing multiple samples or patient records.



```
ic50 = 0.045 # µM

if ic50 < 0.01:
    potency = "High"
elif ic50 < 1.0:
    potency = "Moderate"
else:
    potency = "Low"

print(f"Potency: {potency}")
# Output: Potency: Moderate
```



```
diseases = ["Malaria", "Cholera", "Tuberculosis"]

for disease in diseases:
    print (disease + "is bad")

# Output :
Malaria is bad
Cholera is bad
Tuberculosis is bad
```

# Basic concepts – Mathematical Operations

- Python follows the standard order of operations: BODMAS
- Math operations can be performed directly on variables
- Division (/) returns a decimal value, while floor division (//) rounds down to the nearest whole number
- The math library provides advanced mathematical functions such as square roots, logarithms, factorials, and trigonometry.
- The math library must be imported before use with **import math**



# Using the math library

```
import math
```

```
print(math.sqrt(25))    #Output = 5, sqrt=square root  
print(result)
```

# Order of operations

```
result = 5 + 2 * 3
```

```
print(result)    #Output = 11, multiplication before  
addition
```

# Operators

Addition: '+'

Multiplication: \*

Division: /

Floor division: //

Subtraction: -

Power: \*\*

Modulus: %

## EXERCISE 4: Calculate Vaccination Coverage

Create variables for:

Number of vaccinated children

Total number of children surveyed, then Calculate the vaccination coverage percentage.

# Basic concepts – Functions

- **Functions** are reusable blocks of code used to perform a specific task.
- Functions are defined using the **def** keyword followed by a function name.
- **Parameters** are inputs passed into a function so it can work with different values.
- The **return** statement sends a result back from the function.
- After the function parameters, a colon (:) must be added, and the code inside the function must be indented correctly for Python to recognise it as part of the function.
- Functions help make code cleaner, shorter, and easier to reuse.



```
def calculate_bmi(weight, height):  
    bmi = weight / (height ** 2)  
    return bmi
```

```
result = calculate_bmi(70, 1.75)
```

```
print(result)  
# Output : 22.86
```

# Parts of the function

def – Keyword that defines the function

Calculate\_bmi – function name

Weight, height – parameters

return – sends the answer back

## EXERCISE 5: Calculate Patient Temperature Difference

Create a function called `temperature_difference` that:

1. Takes two temperature values
2. Subtracts the normal temperature (37°C) from the patient's temperature
3. Returns the difference

# End of Module Exercise

## Teaching Good?

### Attempt the exercise below:

#### Challenge 1: The Drug Inventory Manager

You are a lab manager. You have a dictionary of drug stocks (drug name → quantity in mg). Write a function that takes this inventory and a minimum threshold and returns a sorted list of drugs that need restocking, along with the amount remaining.

```
●●●
# Starter code
inventory = { "Ibuprofen": 450, "Metformin": 80,
              "Aspirin": 1200,  "Lisinopril": 30, "Atorvastatin":
              15, "Paracetamol": 900}

def check_stock(inventory, threshold):
    # your code here
    pass
```

#### Pro-tips:

- Try the challenge on your own before depending on an AI/Large language model.
- Always read the error messages and try to resolve it.



# Content

## 1 Course Overview

## 2 Basic Concepts

## 3 Data Analysis with Python

# Loading a Dataset

- Python can be used to load, organise, and analyse datasets.
- Most scientific datasets are stored as CSV or Excel files.
- The **pandas** library is commonly used for working with data in Python.
- A dataset is usually organised into rows and columns, similar to Excel.
- Each column represents a variable, such as age, diagnosis, or laboratory result.



```
import pandas as pd #Imports the pandas library

data = pd.read_csv("patients.csv") # Loads a csv
file named patients.csv
data = pd.read_excel("patients.xlsx") # Loads an
excel file named patients.csv

# Viewing the first rows:
data.head() # displays the first five rows of data

# Viewing the last rows:
data.tail() # displays the last five rows of data

# Getting dataset Information:
data.info() # shows a summary of dataset including
column names, no of entries, missing values and
data types.
```

## EXERCISE 6: Load and explore health data

Download this [dataset](#)

1. Put it in your working folder
2. Load the dataset and use the head, tail, info and other functions to explore it

# Cleaning a Dataset

- Real-world scientific datasets are often incomplete or inconsistent.
- Data cleaning improves the quality and reliability of analysis.
- Common problems include missing values, duplicate records, and incorrect formatting.
- Python provides simple tools for identifying and correcting data issues.
- Clean data is essential for accurate statistics, visualisation, and machine learning.



```
import pandas as pd #Imports the pandas library

data = pd.read_csv("patients.csv") # Loaded dataset

# Check for missing values
data.isnull().sum() # counts missing values in each column

# Removing missing values
data.dropna() # removes missing values

# Filling missing values
data.fillna(0) # replaces missing values with 0

# Removing duplicate rows
data.drop_duplicates() # deletes duplicate rows
```

## EXERCISE 7: Clean health data

From the dataset downloaded in exercise 6,

1. How many missing values are there?
2. Rename a column and change an integer row to a string row (hint: use the `.str()` function)



# Basic Data Visualization

- Data visualisation converts numerical data into graphs and charts that are easier to interpret.
- Visualisations help researchers identify trends, outliers, distributions, and possible errors in datasets.
- Different chart types are used for different kinds of data, such as histograms for distributions and bar charts for categories.
- The matplotlib library is one of the most commonly used Python libraries for creating scientific graphs.



```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("patients.csv") # Loaded dataset

plt.hist(data["Age"])
plt.show() # displays a histogram of the age column
```

## EXERCISE 8: Clean health data

From the dataset downloaded in exercise 6,

1. Create a bar chart using any column and label the axes
2. Create a scatter plot using any column

# Conclusion

You have come to the end of the basic python for life sciences course!

What Next? Continue Practicing

- Practice basic python concepts with problems at [Leetcode](#)
- Download and explore free data sets at [Kaggle](#)

**Questions or Support?**

[Send me a mail](#)

*Thank  
you*

